

Question 1 — What is normalisation?

1. What is Normalisation?

Normalisation is the process of organising data in a database into structured and related tables to reduce data duplication and improve data integrity. It involves dividing large tables into smaller tables and creating relationships between them. The purpose of normalisation is to ensure that data is stored efficiently and consistently, making the database easier to maintain and update.

For example, instead of storing a customer's details repeatedly every time they place an order, the customer's information can be stored in a separate customer table and linked to their orders using a key.

Easy way to remember it

Normalisation = organising database data so that information is not unnecessarily repeated.

Question 2 — When is a table in 1NF?

A table is in First Normal Form (1NF) when each column contains atomic values, meaning that each cell contains only one value. The table should not contain repeating groups or multiple values stored in a single field. Each row should also be uniquely identifiable using a primary key.

For example, a customer table should not store multiple phone numbers in one cell separated by commas. Each field should contain a single value, allowing the data to be stored and managed consistently.

Easy way to remember

Think:

1NF = ONE

ONE value in each cell.

That's the main idea.

Question 3 — When is a table in 2NF?

A table is in Second Normal Form (2NF) when it is already in First Normal Form (1NF) and all non-key attributes depend on the entire primary key. A table is not in 2NF if a non-key attribute depends on only part of a composite primary key. This is known as a partial dependency.

For example, if a table has a composite primary key consisting of Student ID and Course ID, the student's name should not be stored in the same table if it only depends on Student ID. Similarly, the course name should not be stored there if it only depends on Course ID. These attributes should be moved into separate tables to remove the partial dependencies.

The easiest way to remember 2NF

Think:

"Does this information need ALL of the primary key?"

If the answer is **no**, you probably have a **partial dependency**.

Question 4 — When is a table in 3NF?

A table is in Third Normal Form (3NF) when it is already in Second Normal Form (2NF) and has no transitive dependencies. A transitive dependency occurs when a non-key attribute depends on another non-key attribute instead of directly depending on the primary key.

For example, if Student ID determines Department ID and Department ID determines Department Name, then Department Name has a transitive dependency on Student ID. To achieve 3NF, the department information should be placed in a separate table and linked using a key.

The easiest way to remember 3NF

Think:

3NF = Remove information that depends on another non-key.

Or even easier:

3NF = No middleman.

Question 5 — Identify the dependencies

The table has a composite primary key consisting of INV_NUM and PROD_NUM.

Partial Dependencies

The following attributes depend only on part of the composite primary key:

INV_NUM → SALE_DATE

The sale date depends only on the invoice number because all products on the same invoice have the same sale date.

PROD_NUM → PROD_LABEL

The product label depends only on the product number because each product number identifies a specific product.

PROD_NUM → VEND_CODE

The vendor code depends only on the product number because each product is associated with a specific vendor.

PROD_NUM → PROD_PRICE

The product price depends only on the product number because each product has its own price.

Full Dependency

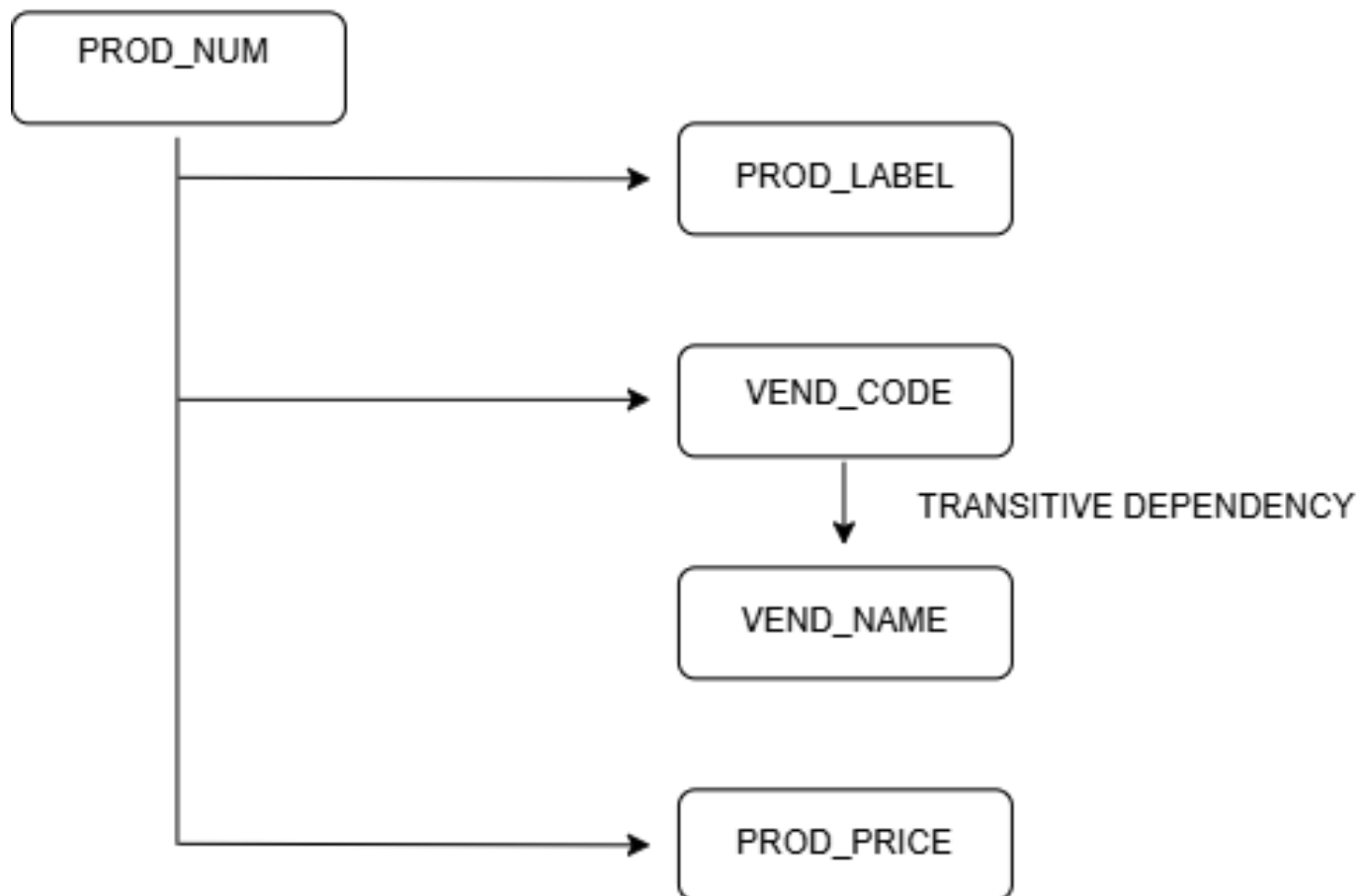
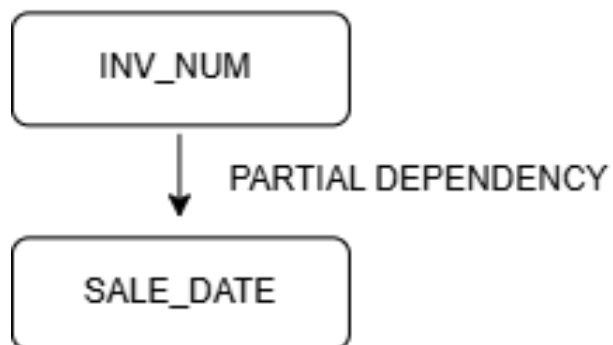
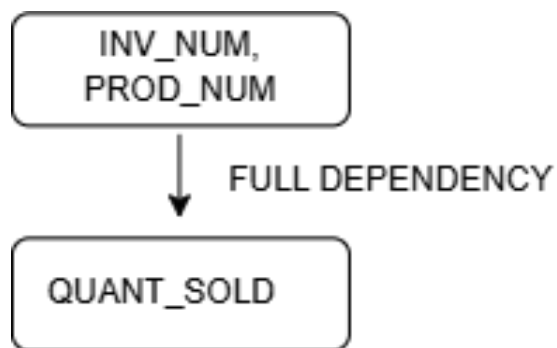
(INV_NUM, PROD_NUM) → QUANT_SOLD

The quantity sold depends on both the invoice number and the product number. Both values are required to determine how many units of a particular product were sold on a specific invoice.

Transitive Dependency

PROD_NUM → VEND_CODE → VEND_NAME

The vendor name depends on the vendor code, while the vendor code depends on the product number. Therefore, the vendor name has a transitive dependency on the product number.



Question 6 — Remove Partial Dependencies

To achieve Second Normal Form (2NF), the partial dependencies identified in Question 5 are removed by separating the data into different tables. The invoice information is placed in an INVOICE table, the product information is placed in a PRODUCT table, and the information that depends on the complete composite key is placed in an INVOICE_PRODUCT table.

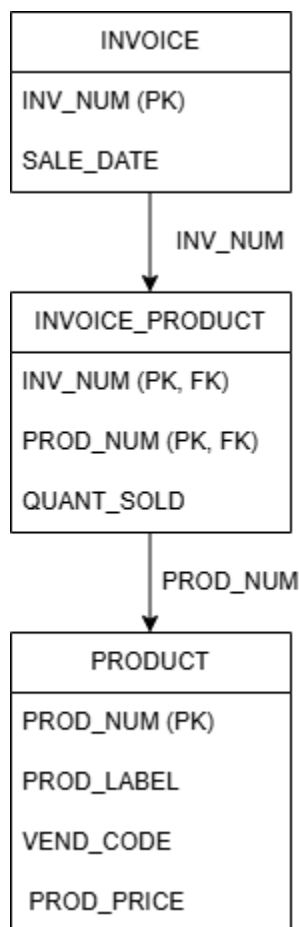
The resulting tables are:

INVOICE: INV_NUM (PK), SALE_DATE

PRODUCT: PROD_NUM (PK), PROD_LABEL, VEND_CODE, PROD_PRICE

INVOICE_PRODUCT: INV_NUM (PK, FK), PROD_NUM (PK, FK), QUANT_SOLD

This removes the partial dependencies because each non-key attribute now depends on the appropriate complete primary key.



Question 7 — Remove Transitive Dependencies

To achieve Third Normal Form (3NF), the transitive dependency identified in Question 5 must be removed. The transitive dependency is:

$\text{PROD_NUM} \rightarrow \text{VEND_CODE} \rightarrow \text{VEND_NAME}$

VEND_NAME depends on VEND_CODE rather than directly on the primary key of the PRODUCT table. Therefore, VEND_CODE and VEND_NAME are separated into a new VENDOR table.

The final tables are:

INVOICE: INV_NUM (PK), SALE_DATE

PRODUCT: PROD_NUM (PK), PROD_LABEL, VEND_CODE (FK), PROD_PRICE

VENDOR: VEND_CODE (PK), VEND_NAME

INVOICE_PRODUCT: INV_NUM (PK, FK), PROD_NUM (PK, FK), QUANT_SOLD

This removes the transitive dependency and results in a database structure that is in Third Normal Form (3NF).

